Software Engineering II

Workshop 1



Zero_One:

Julian Ramirez Diaz

1/10/2026

1. **Requirements Documentation**
    - List and describe the main functional requirements (what the system should do).
    - List and describe the main non-functional requirements (quality attributes, e.g., usability, performance, security, scalability).

2. **User Stories**
    - Write user stories for different roles (e.g., end-user, admin, manager).
    - Use the format: As a [role], I want to [action] so that [benefit].
    - Each story should include acceptance criteria in the format: Given [context], When [event], Then [outcome].

3. **User Story Mapping**
    - Create a visual map or table showing how user stories are grouped and prioritized for development.
    - Example resource: https://plan.io/blog/user-story-mapping/
    - Releases of the project

4. **Project Requirements Checklist**
    - ☑ ~~Clear and complete requirements (functional and non-functional).~~
    - ☑ ~~Well-defined user stories and acceptance criteria.~~
    - ☑ ~~User story mapping for planning.~~
    - ☑ ~~Organized and referenced documentation.~~

1. Requirements Documentation

1.1. Functional Requirements

ID	Requirement	Description	Actor
FR-01	User Registration	Players can create an account using an email and password.	Player
FR-02	Riot Account Linking	Players can link their League of Legends account using their Riot ID (GameName#TAG) and region.	Player
FR-03	Data Fetching	The system fetches the player's current rank and recent match history from the Riot API.	System
FR-04	Champion Recommendations	The system guides the player through choosing a champion to “Main” based on what the player is good at (according to their data).	System
FR-05	Build recommendations	The system shows a build for the current patch and champion of the player.	System
FR-06	System monitoring	The admin has access to a dashboard of Riot API usage versus limits, errors and import queue, with alerts.	Admin

1.2. Non-Functional Requirements

ID	Category	Requirement	Measure / Target
NFR-01	Performance	Page load times.	Main pages load in under 3 seconds on a standard connection.
NFR-02	Security	Password protection.	Passwords must be hashed and never stored in plain text.
NFR-03	Compliance	Riot API limits.	The system must respect Riot API rate limits and cache data to avoid being blocked.

ID	Category	Requirement	Measure / Target
NFR-04	Data freshness	Game data matches the live patch	Data is updated within 48h of a patch release; builds may take some time to adjust to new METAs.

2. (Some) User Stories

2.1. US-01: Create Account

2.1.1. As a Player, I want to create an account so that my progress and preferences are saved.

2.1.2. Acceptance Criteria:

2.1.2.1. Given that I am on the registration page,
2.1.2.2. When I enter a valid email and a password of at least 8 characters,
2.1.2.3. Then my account is created and I am redirected to the dashboard.

2.2. US-02: Link Riot Account

2.2.1. As a Player, I want to link my Riot ID so that the system can fetch my actual match data.

2.2.2. Acceptance Criteria:

2.2.2.1. Given that I am logged in and on the settings page,
2.2.2.2. When I enter my valid Riot ID (Name#TAG) and select my region,
2.2.2.3. Then the system verifies the ID with the Riot API and saves it to my profile.

2.3. US-03: View Recommendations

2.3.1. As the System, I want to be able to show the Player champion recommendations so that they can decide who to play next.

2.3.2. Acceptance Criteria:

2.3.2.1. Given that the Player's Riot account is linked and data is fetched,
2.3.2.2. When the player navigates to the "Recommendations" page,
2.3.2.3. Then I show a list of at least 3 champions that the player might feel comfortable playing, according to their statistics.

2.4. US-04: Monitoring the System

2.4.1. As an Admin, I want to be able to see a dashboard of Riot's API usage so that I stay within Riot's rate limits and detect problems early.

2.4.2. Acceptance Criteria:

2.4.2.1. Given that i am visualizing the Admin Dashboard
2.4.2.2. When some time passes
2.4.2.3. Then I see requests per minute versus the allowed limit, error rate and queued imports, refreshed at least every minute.

3. User Story Mapping

Activity / Backbone	1. Onboard & Connect	2. Discover & Play	3. System Administration
User Tasks	Sign up, Link Riot ID	View Recommendations, View Builds	Verifying the system stays free of errors
Release 1 (MVP)	US-01, US-02	US-03	//
Release 2 (V2)	//	//	US-04

Releases of the Project

- **Release 1:** MVP (Minimum Viable Product)
  - Goal: Allow a player to register, link their Riot account, and see basic champion recommendations.
  - Stories included: US-01, US-02, US-03.
  - Definition of Done: The Player can complete the flow from registration to seeing recommendations without errors.
- **Release 2:** Administration & Enhancements
  - Goal: Give admins the power to check the current status of the system and the API.
  - Stories included: US-04 (and future stories for detailed builds).
  - Definition of Done: Admins can successfully verify the app is within the limits of Riot's API.

4. References

➔ **This document was made with the help of Claude's AI and Qwen's AI, everything was reviewed properly and not pasted carelessly.**

➔ Riot Games. Riot Developer Portal, API documentation, Data Dragon (static game data) and API policies. https://developer.riotgames.com

➔ plan.io. User Story Mapping. https://plan.io/blog/user-story-mapping/

➔ itero.gg. Inspiration for the product vision